PRIVATE OPERATING MANUAL

Business Intelligence Dashboard

How to work every feature, and how to learn every field

Starting · Workspaces and modes · Running your freelance work · Data tools · The learning system · Twelve field chapters · AI and privacy · Troubleshooting

Prepared for Lamarcus · October 2026 · matches install zip 54

Contents

About this manual 4

Part 1: Starting and finding your way around 6

Part 2: Workspaces, modes and the safety model 9

Part 3: Running your freelance work 11

Part 4: Working with data 15

Part 5: The learning system 19

Part 6: The fields, one chapter each 23

Field 1: Data Analyst 24

Field 2: Data Scientist 27

Field 3: Machine Learning 30

Field 4: Data Engineering 33

Field 5: AI Engineering 35

Field 6: Project and Product Management 38

Field 7: Systems Analyst 41

Field 8: Full Stack Developer 44

Field 9: Cybersecurity 46

Field 10: Network Engineer 49

Field 11: IT Repair and IT Specialist 52

Field 12: Computer Engineer 55

Part 7: AI, privacy and trust 58

Part 8: When something goes wrong 60

Appendix A: Every page and where it lives 62

Appendix B: Quick glossary 64

Appendix C: Your first month 3

About this manual

This is the operating manual for your private Business Intelligence dashboard. It explains how to start it, how every page works, how to run your freelance work with it, and how to use it to learn each of the twelve fields built into it. It is written for you, Lamarcus, and for nobody else.

Edition: October 2026, matching install zip 54 (it includes everything from zips 1 to 53).

The five rules that never change

1. **The tool is private.** It is never shown to clients and clients never log in. Clients only receive deliverables you choose to hand over: an invoice, a summary, a report, a handoff pack.
2. **Nothing is sent for you.** Reminders, quotes, client messages and AI drafts are text for you to read and send yourself. The app does not email, post or publish on its own.
3. **AI can be wrong.** Every AI draft is a draft. Check the numbers, check the facts, and never paste client secrets into a question.
4. **Practice data is labelled.** The Practice workspace holds made-up data. Practice work is never presented as real work, and the Sample work page stamps every item as a practice sample.
5. **Only your own systems.** Network, security and computer tools are for equipment you own or have written permission to touch.

How to read this manual

- **New to the dashboard?** Read Part 1 (start it, learn the layout) and Part 2 (the two workspaces and two modes), then jump to the field you want.
- **Running a job?** Part 3 covers the business side: jobs, the timer, invoices, money, leads, quotes.
- **Studying?** Part 5 explains Learn mode, and Part 6 gives one chapter per field with a week-by-week route.
- **Something broke?** Part 8 has the troubleshooting table and the install commands.

Menu names are written in **bold**. A path such as **Settings > AI and privacy** means: open Settings, then open the tab called AI and privacy. Page addresses (for example `/lab`) are given in code type so you can type them after `localhost:3012`.

What is in the box

Area	What it gives you
Work mode	Home with today's list, Clients and jobs, Money, Leads, data tools, dashboards, and one dashboard per field

Area	What it gives you
Learn mode	Career tracks with how-to manuals, quizzes, spaced review, the glossary, labs and practice pages
Twelve field dashboards	Analyst, Data Scientist, Machine Learning, Data Engineering, AI Engineering, Project and Product, Systems Analyst, Full Stack, Cybersecurity, Network, IT repair and Computer Engineering, each with working tools, an advanced lab, and an AI agent
Extra fields	BI, DevSecOps, Cloud, Health data, Science and research, Business consulting, Marketing and design
Assistants	One AI agent per field, a three-AI work team, an Assistant desk to approve drafts
Safety	Practice and Real workspaces, backups, privacy modes, host and origin checks

Part 1: Starting and finding your way around

What the dashboard is

The dashboard is two programs that run on your own computer. The **backend** (a FastAPI program on port 8020) holds your data, does the calculations and talks to the AI services. The **screen** (a Next.js web app on port 3012) is the part you see in the browser at `http://localhost:3012`. Nothing is hosted online, nobody else can reach it, and there is no login because only you use it.

Starting it with the launcher

In your project folder (`C:\Users\Guito\Business Intelligence DB\`) there is a file called **Start BI Dashboard.bat**. Double-click it and a menu appears:

Choice	What it does
1 Start (everyday)	Starts the backend and the screen inside WSL, waits until both answer, then opens the dashboard in your browser. Scanned-file reading (OCR) works in this mode.
2 Start with live network capture (administrator)	Windows asks for administrator permission. The backend then runs in Windows (using the `.venv-win` environment) so Npcap can see your real network card. The screen still runs in WSL. Scanned-file reading works in this mode only if Tesseract for Windows is installed too (see Part 4).
3 Stop everything	Stops the backend and the screen, in WSL and in Windows.
4 Open the dashboard in my browser	Opens `http://localhost:3012` without starting anything.
5 Quit	Closes the menu.

Notes:

- If the screen has never been built, the launcher builds it first. That takes a few minutes the first time.
- Only one backend can run at a time (they share port 8020). If you switch between choice 1 and choice 2, use choice 3 in between.
- The launcher never connects to the internet and never sends anything.
- For a desktop shortcut: right-click the file, choose **Send to**, then **Desktop (create shortcut)**.
- The launcher was written and proofread but not run on Windows by the person who wrote it. If a window shows an error, copy the message and report it.

Starting it by hand

If you prefer terminals, start the backend in one and the screen in another.

```
# Terminal 1: backend (WSL)
cd "/mnt/c/Users/Guito/Business Intelligence DB/backend"
source ../.venv/bin/activate
python -m uvicorn app.main:app --port 8020

# Terminal 2: screen (WSL)
cd "/mnt/c/Users/Guito/Business Intelligence DB/frontend"
npm run build
npx next start -p 3012
```

A new backend feature needs a backend restart. A new screen feature needs `npm run build` again.

Installing an update zip

Each update arrives as `bi_updateN.zip` in your project folder and contains every earlier update. The routine, from the WSL terminal in VS Code:

```
cd "/mnt/c/Users/Guito/Business Intelligence DB"
ls bi_updateN.zip
rm -rf ~/tmp_bi && mkdir -p ~/tmp_bi
unzip -oq bi_updateN.zip -d ~/tmp_bi
cp -r ~/tmp_bi/bi/. .
rm bi_updateN.zip
source .venv/bin/activate
pip install -r backend/requirements.txt
git add -A && git commit -m "Install zip N" && git push
cd frontend && npm run build
```

Then restart the backend. Your own data is not in the zip, and `.gitignore` keeps real uploads and the warehouse files out of git. Before you push, run `git status` once and make sure no real client file is listed.

The screen layout

Every page has the same frame.

- **Top bar:** the Practice / Real workspace switch, the Work / Learn mode button, Settings, and **Save page as PDF**.
- **The top box:** a single text box under the title area. Type a question or an instruction and click **Go** (it suggests dashboards), or click **Start as job** (it creates a job from your words and opens the right field). More on this in Part 3.
- **The pinned job bar:** when a job is open, a bar stays under the top box on every page with the timer, steps, a quick note, a Field switcher, Switch job and Close job.

- **The menu row:** in Work mode: Home, Clients, Money, Leads, Data, Dashboards, Fields, More. In Learn mode: Tracks, Quiz, Review, Glossary, Labs and practice, Fields, How-to guides.
- **Command palette:** press **Ctrl+K** (Cmd+K on a Mac) on any page to search pages and actions, move with the arrow keys, press Enter to go, Escape to close.
- **Dark and light themes:** the theme button in the top bar. Dropdowns are readable in both.

Your first ten minutes

1. Start the dashboard with the launcher, choice 1.
2. Look at the top bar and confirm it says **Practice**. You are on made-up data.
3. Open **Home**. Read the Today panel and the Open a dashboard list.
4. Open **Fields** and pick a field you want to study. Notice the three views on a track dashboard: How-to manual, Tools, AI agent (Learn mode) or Tools with the agent beside them (Work mode on a wide screen).
5. Open **Settings > AI and privacy** and check which AI keys are set up. Without keys everything except the AI agents still works.
6. Press Ctrl+K, type "quiz", press Enter. Take one round.
7. Switch to **Real** only when you are ready to put real client data in (Part 2 explains the difference).

Part 2: Workspaces, modes and the safety model

Practice and Real workspaces

	Practice	Real
Data file	`bi_warehouse.duckdb`	`bi_warehouse_real.duckdb`
Starts with	Demo data	Nothing
AI	On	**Off** until you turn it on
Backups	Manual	Manual, plus a daily automatic backup, plus a safety backup before anything destructive

Switch with the **Practice / Real** buttons at the top. The page reloads so every number comes from the other file. Your study progress, ML run history, settings, saved sources and the activity log live in a small separate file shared by both workspaces, so switching never loses your learning record.

When to use which. Practice is for learning, for the test-data jobs, and for experiments that might break things. Real is for paying clients. Compare (**Practice vs Real**, in Learn mode) shows the two side by side and only reads.

Test data in Practice. A script fills Practice with made-up clients, jobs, invoices and tables so every page has something to show. Twenty of the twenty-four practice jobs come with made-up files, a plain brief, a "what done looks like" list and a hidden answer key computed from the same data (open the job and look for **Practice materials**). IT repair and Cybersecurity practice jobs have no files: use your own equipment.

Work mode and Learn mode

The mode button in the top corner switches the whole menu. The choice is remembered in the browser.

- **Work mode** (the default) shows only what earns money: Home, Clients, Money, Leads, Data, Dashboards, Fields and More. Learning pages are hidden. If you open one by a link you see a prompt to switch. Nothing is deleted.
- **Learn mode** shows Tracks, Quiz, Review, Glossary, the labs, the practice pages, the how-to guides and the Fields menu with its study tabs (how-to search, simulators, cheat sheets, exam prep).

A useful habit: study in Learn mode with Practice selected; work in Work mode with Real selected. The top bar always shows you which pair you are in.

Fields

Fields opens a main screen of cards grouped Data, Tech, AI and Research, and Business. Each card opens that field's own dashboard: its tools, plus the services you can sell in that field with prices. IT repair, Network and Cybersecurity keep their track dashboards; the rest open `/field/<name>`.

In **Settings > General** there is a **Mode and my fields** section where you tick only the fields you sell. The Fields menu then shows only those.

What protects your data

- The backend listens on `127.0.0.1` only. Other devices on your Wi-Fi cannot connect.
- **Host check:** requests that arrive under any name other than `localhost` or `127.0.0.1` are refused (this blocks a trick called DNS rebinding).
- **Cross-site writes are refused and logged:** another website cannot make your browser change your data.
- **Security headers** and a Content-Security-Policy on the built screen.
- **Secrets** live in `backend/.env`. Update zips never include it, and the build fails if it finds a key in a text file.
- **Daily backup of Real data** runs whenever the app is running (at most once every 20 hours), even if you are in Practice.
- **AI privacy modes** per workspace: Off, Summaries only, Full, plus blocked columns (Part 7).

There is **no login** and **no encryption of the files on disk**. Keep the dashboard on your own machine and turn on Windows disk encryption (BitLocker) if the laptop could be lost. Do not expose it to the internet. If you ever host it, add authentication first (the app supports an owner password for that case).

Limits to keep in mind

Up to 1,000,000 rows and 300 columns per table, 50 MB per upload. One person, one machine. Scheduled refreshes run only while the app is running. The Docker files in the repository have not been tested.

Part 3: Running your freelance work

This part is for Work mode with the **Real** workspace. Every step can be rehearsed in Practice first.

The one-place job flow

Everything starts from the **top box**.

1. Type the job in your own words: for example "Field: network, Rivera Dental, office Wi-Fi keeps dropping, quote needed".
2. Click **Go** to get suggested dashboards, or click **Start as job**. Start as job picks a client and a field from your words, creates the job with a first step, and opens that field's dashboard. The field guess uses a "Field:" prefix in the title first, then keywords. You can change it.
3. The **pinned job bar** now sits on every page. It holds:
   - the **timer** (start and stop; hours go to the job as billable time),
   - the job's **steps** (standard steps for each field are added for you),
   - a **quick note**,
   - a **Field switcher**,
   - **Switch job** and **Close job**.
4. One-key shortcuts work when you are not typing in a box: **T** timer, **D** done and next step, **N** note, **A** add step, **C** close, **?** show the list.
5. Work the job with the field's tools (Part 6) and the **Agents do the job** card, which lets the field's agent and the three-AI team draft the work. Drafts only.
6. When every step is done, the job page and Today say so. **Close job** creates the invoice (hours, quoted price, or none) and a client-ready summary page.

Fields the job flow understands: IT repair, Networking, Cybersecurity, Data analyst, Data scientist, Machine learning, AI engineering, Engineering, Data engineering, Full stack, Project management, Systems analysis.

Home and the Today panel

Home in Work mode is your morning screen.

- **Today** lists late invoices (with a reminder already written for you to copy; nothing is sent), follow-ups due, late jobs, overdue work items, a timer left running over two hours, jobs ready to close, and a backup reminder.
- **Open jobs** shows each open job with a **Work on it** button that pins it and opens its field.
- **Quick starts** are one-click shortcuts.
- **Open a dashboard** lists every dashboard.

- **Today's briefing** and **System health** summarise your data and the app's own state. The briefing can also be set up as a morning email (Settings > Alerts and reports > Morning briefing).

Clients (`/clients`)

The page for your own freelance work: clients, jobs, hours, invoices, follow-ups, and a handoff pack for a paying client.

- **Add a client** with contact details, notes and a default hourly rate.
- **Jobs** belong to a client. Each job has a field, a status, steps, work items, hours, an invoice and a closing note.
- **Recurring jobs** (per client): weekly or monthly, created with their steps once per period while the app is running.
- **Client messages:** ten message types with free templates and an optional AI version (quote follow-up, scope change, late invoice and so on). You copy the text and send it yourself.
- **Paste-in client message:** paste what the client wrote; the app applies rules, drafts a quote and proposes steps.
- **Handoff pack:** the document you give a paying client at the end. Nothing is sent from the app.
- **Client notes** stay in the app. Client names are removed before anything is sent to an AI service.

One page per job (`/job?id=...`)

Hours, the unbilled amount, work items, fixes recorded for that client, and invoices. **Practice materials** (for practice jobs) show the made-up files, the brief, "what done looks like" and a hidden answer key you open only after you have tried. Each work item has a timer: hours go to the linked job as billable time and onto the invoice. A timer left running is capped at 12 hours. Work items not linked to a job keep their time but it is not billed.

The Assistant desk

AI help for your jobs, but with a human gate.

- **Approval queue:** every AI draft lands here. Edit it, accept it, or discard it.
- **Job descriptions per assistant** tell each assistant what its role is.
- **Activity log with a used rate** shows how often you actually accept drafts.
- **Your saved examples** rank first when the assistant drafts, with client names removed.
- **Closing note:** a note drafted for each job that only reaches the client once you accept it.

The work team

In the pinned job bar, type the job and choose **Draft with the team**. Gemini, OpenAI and Claude each draft it, then Claude judges the three and writes one final draft. It lands in the Assistant desk. It is off by default and has its own daily cap. For IT, Network and Cybersecurity it gives checklists and advice only. In Real, only your instructions and pasted material are seen, with client names removed and tables summarised locally.

Money (`/money`)

Who owes you, what each job really earned, and your expenses, mileage and tax set-aside.

- **Receivables:** unpaid and late invoices, with the days late.
- **Job profit:** what each job earned after your time and expenses.
- **Expenses and mileage:** enter them as they happen.
- **Tax set-aside:** a running estimate of what to put aside. It is an estimate, not tax advice; check the rate with an accountant.
- **Reminders are drafts.** You copy them and send them yourself.

Leads (`/leads`)

Where each lead came from, what you quoted and whether you won it, so you can see which sources are worth your time. A weekly **who to contact** list is drawn from open leads and quiet clients. Message templates here use the same ten client-message types.

Quotes, prices and services (Settings)

- **Settings > Business and services:** your business details and **service hourly rates**.
- **Settings > Quotes and prices:** the **quote builder** and a researched **Henderson and Las Vegas price list**. Hourly rates in the service catalog start from that list. Change any rate whenever you like.
- On each field page, **Services you can sell in this field** shows the services, the price range and the hourly rate.

Sample work page (`/samples`)

A separate downloadable page of **labelled practice work** for prospects. The rules are enforced in code:

- every item is stamped "Sample · practice work · made-up data";
- email addresses and phone numbers are rejected in item text (your own contact line is the only exception);
- review and testimonial wording is rejected;
- the page makes no outside requests and has no scripts, and its text is escaped;
- quote totals are computed on the server;
- **Download** gives you `sample-work.html`.

Nothing is published or sent from the dashboard. If you share the file, you are sharing practice samples, and that is what it says on every item.

Settings tabs

Tab	What is in it
General	Practice or Real switch, appearance and theme, **Mode and my fields**, **My projects** (links to Eivanta Analytics and Project AIDA), and **Install on this device** (add the dashboard to your desktop so it opens like an app)
Business and services	Business details and **service hourly rates**
Quotes and prices	Quote builder and the Henderson and Las Vegas price list
AI and privacy	One card per workspace: privacy mode, blocked columns, provider status and daily caps
Data and backups	Backups and restore, the business backup, and **Export everything**
Alerts and reports	Alerts on your Real data with email through SMTP, and the **Morning briefing**
Security and device	Sign-in password (for when it is hosted) and **Laptop and travel** options
Scanned files and capture	OCR status and test box; the live network capture card
Legal and compliance	Compliance library (HIPAA, SOC 2, PCI and more) used by the security and IT agents

Backups in practice

Settings > Data and backups. A backup is a copy of the database file taken in milliseconds while the connection is closed. Restoring first saves the current state as a "safety" backup, so a restore can be undone. The app keeps the latest 14 backups (change this) and the latest 10 safety copies. **Download** a backup to store it elsewhere. **Backups sit on the same disk as the data**, so copy them to an external drive or a cloud folder now and then. You can also run `installer/backup_now.bat` at any time, even when the app is not running.

Part 4: Working with data

Getting data in

There are four doors, from simplest to most automatic.

1. My data (`/data`)

Pick a CSV, Excel (.xlsx) or JSON file. Before anything is saved you see the detected columns and types.

- Separators (comma, semicolon, tab, pipe) are detected.
- Numbers like `$4,200.50` become numbers; codes like `02134` stay text.
- Dates are read as `YYYY-MM-DD` or `MM/DD/YYYY`. Day-first dates such as `31/12/2026` are not guessed, to avoid silently swapping day and month.
- Choose **Feed the operations dashboards** to point your columns at the main dashboard: say which column is the date, the entity, revenue, cost and units. If your file has no such column, type a fixed value such as `ALL`.
- You can look at the table, chart it, and use it in the labs. Nothing leaves your computer.

File rules for every Upload button. One shared set of plain-English rules applies everywhere: an unfinished download (`.crdownload`), an empty file, an old `.xls`, an `.evtx`, a zip or a wrong type each get a clear message. Text readers (security logs, IT events, theme finder) also take Excel, PDF and Word by converting them first. Photos and scanned PDFs are read through OCR when Tesseract is installed (below).

2. Data Intake (`/intake`)

Upload a file and a **supervisor** runs it past six agents in order (security, engineering, analyst, science, ML, BI), stopping if a gate fails. Nothing is overwritten. After a run, **Where do you want to work on this file?** ranks dashboards from the file's columns, types and name. Click one and it opens with a **Your uploaded file is ready here** banner and a button that starts that dashboard's agent on the file. The agent sees column names and the first rows only, with personal-data columns left out.

3. Sources (`/sources`)

Save where data comes from once, then refresh it on demand or on a schedule.

Source type	Notes
File in a folder	Files in `backend/data/inbox/` or any folder in `BI_SOURCE_DIRS`. Paths outside those folders, `..` tricks and symlinks are refused.
Web link	An `http(s)` CSV, JSON or Excel URL. For a private API put the token in `backend/.env` (for example `MY_API_TOKEN=...`) and type only the variable name. Redirects are followed; downloads are capped at 50 MB.

Source type	Notes
Database query	Postgres (put `PG_URL=postgresql://user:pw@host/db` in `.env` and type `PG_URL`) or a SQLite file in an allowed folder. Only a single `SELECT` is accepted and the connection is read-only.

What makes a refresh safe: the new data is read and type-checked completely before the old table is touched, then swapped in one step; a failure leaves the old table as it was; a "replace" that would leave under 50% of the old rows is refused; in Real a safety backup is taken first; every run is recorded in the source's history and the activity log. Scheduled refreshes run inside the app, for the workspace that is open, only while the app is running.

4. Paste intake

Several dashboards have a paste box: paste text (a log, a config, a ticket, a list) and the tool reads it directly.

Checking the data

Data quality (`/quality`)

Nine checks you run before you trust any dashboard: nulls, negatives, orphans, duplicates, date gaps and more, each with sample rows so you can see the problem. Garbage in, garbage out. Make this the first stop on any new file.

Dataset hub (`/hub`)

Everything you have imported, with a health check, notes, tags and quality rules. A small data catalogue of your own.

Schema and star schema (`/schema`)

How the dashboard's data is modelled: one fact table in the middle, dimensions around it. The SQL Lab and the Systems Analyst track both use it.

Looking at the data

Main dashboard (`/bi`)

KPIs, filters, a trend with a forecast, insights, drill-down, and **Excel and PDF reports** of the current view. The **Chart gallery** below the entity chart has a dropdown of twelve charts over your filtered data: pie, donut, treemap, ranked bar, Pareto, radar, funnel, stacked area, waterfall, histogram, box plot and bubble. Pick the measure (revenue, cost, profit, units, duration) and what to group by. Remember: margin is a ratio of sums (total profit divided by total revenue), never an average of row percentages.

KPI builder (`/kpis`)

A metric measures; a KPI ties a metric to a goal. Define the metric, a target, a direction (higher or lower is better), a window and a warning band. The page shows which KPIs are on track.

Business dashboards (`/packs`)

Pick an industry and a department. **Load sample data** to see it, or import your own CSV. Use it to show yourself what a finished dashboard looks like for a given business before you build one.

Glossary (`/glossary`)

One agreed definition per metric, with the SQL behind it. Every example is tested against the data model.

Building and connecting

- **Pipelines (`/pipelines`):** chain tasks with dependencies, retries and a schedule. A small take on Airflow or Prefect.
- **Pipeline monitor (`/pipeline`):** the bronze, silver, gold medallion run with DuckDB and Parquet. Run it, read the quarantine and health checks, and prove it is idempotent (running twice gives the same result).
- **Workflow builder (`/workflow`):** chain simple steps over a table. The app writes the SQL and the pandas for you and shows the rows that survive each step. A good way to learn SQL by seeing it generated.
- **Knowledge search (`/knowledge`):** search the project's docs and every track manual.

Watching and logging

- **Log explorer (`/logs`):** paste or upload a log, search it, and see what the detection rules found. A small take on Splunk or Kibana for one file at a time.
- **Ops monitor (`/ops`):** how the app itself is doing: requests, slow endpoints, database files, scheduled work, and AI calls.
- **Activity log (`/activity`):** everything that changed data or left the app: imports, exports, emails, AI questions, refreshes, backups. Newest first. Check it whenever you wonder what the app did.
- **Alerts:** in Real the app can check for late data, quality failures and KPI breaches and email you through the SMTP settings.
- **Triage checklists (`/checklists`):** step-by-step lists for a slow database, a messy network rack and a huge data file. Print one, or tick it off on a job.

Scanned files (OCR)

Photos and scanned PDFs can be turned into text. **Settings** > **Scanned files and capture** shows whether OCR is ready and has a test box.

- It needs **Tesseract** on the computer that runs the backend. Your Tesseract is installed in **WSL**, so OCR works when you start with launcher choice 1. The Windows backend (launcher choice 2, used for live capture) finds Tesseract by itself once you install the Windows version from the UB Mannheim build (default folder `C:\\Program Files\\Tesseract-OCR`, with English ticked). No PATH change is needed. If you put it elsewhere, set `TESSERACT_CMD` in `backend/.env` to the full path of `tesseract.exe` and restart the backend. With Tesseract in both places, both launcher choices can read scans.
- Once available, every Upload button reads png, jpg, tif, bmp and webp photos, and scanned PDFs (up to 20 pages) fall back to OCR when they have no text.
- OCR is not perfect. Check numbers against the original, especially on receipts and invoices. The tool returns a confidence figure per read.
- The file stays on your computer; reading does not send it anywhere.

Live network capture

Settings > Scanned files and capture > Live network capture. This records packet headers on your own network card so you can read them in the Network and Cybersecurity dashboards or in Wireshark.

- **Requirements:** Npcap installed, the backend running in Windows as administrator (launcher choice 2), and your **own** network only.
- **Limits:** 60 seconds and 5,000 packets per run, a confirm box before it starts, headers only by default (the first 128 bytes of each packet), and a filter box that accepts only safe filter characters.
- **Result:** a summary (packets, protocols, top talkers) and a **Download capture (.pcap)** button. Packet records keep the real packet length even though only 128 bytes are stored, so Wireshark sees honest sizes. Open the file in Wireshark for detail.
- Capturing other people's traffic without permission is not allowed and not what this is for.

Part 5: The learning system

Learn mode turns the dashboard into a study desk. This part explains each piece and gives a way to combine them.

Tracks (`/tracks`)

Career tracks lists the paths, each with real tools and a learning path through this project. Open one and you get a **track dashboard** with three views:

View	What it is
How-to manual	An ordered playbook. Each step says what to do, how, the tool to open (its button jumps to the right tab), how you know you are done, and the usual mistakes. Ticking steps is your own record.
Tools	The working tools for that field (Part 6).
AI agent	A chat for that field. It can read a summary of your workspace's numbers, run read-only SQL, explain the manual step you are on, and draft things for you to approve.

On the right of a track dashboard you also see the **Learning path** (an ordered list of pages, notebooks and files, each with a tick box and a one-line reason) and the **Portfolio projects** for that field. The header shows how many learning-path steps you have ticked.

A **search bar** sits at the top of every field page. Type a question ("which risk is biggest?") or an instruction ("draft five requirements for a billing portal", "take me to the schedule tab").

How the agents behave

- They can read, calculate and **draft**. They cannot change anything by themselves. A draft appears as a card with an **Add** button; the existing, validated API does the write only when you press it. Links open only when you click.
- They follow the workspace's AI privacy mode (Part 7).
- Questions are routed by difficulty: simple to Gemini, medium (comparisons, trends, summaries, drafts) to OpenAI, complex (ML, deep learning, NLP, statistics, math, code, forecasting, multi-part or "why" questions) to Claude. If a provider has no key, has hit its daily cap or errors, the next one is tried. Each reply shows the tier and provider it used.
- A short "yes" or "continue" stays on the same model, topic and reference material as the answer before it.
- Every reply can show its **trace**: the tools used and what each returned.
- They do not learn between questions. For the IT, Cybersecurity and Network agents, the app searches your **fix log** for similar past fixes and hands up to three to the agent as notes, but only when AI mode is Full.

Quiz (`/quiz`)

Exam prep from your class topics: ten questions per round with an explanation after each answer. Each track also has a checkpoint quiz at the end of its manual. Quizzes for the newest advanced labs are not written yet.

Review (`/review`)

Spaced repetition. Questions you miss come back soon; ones you know come back later. A few minutes a day beats a long cram. Open Review at the start of a study session before anything new.

Glossary (`/glossary`)

The metric vocabulary with the SQL behind each term. Use it when a track page uses a word you cannot explain in one sentence.

Labs

Lab	Address	What you do there
SQL Lab	`/lab`	Practice SQL against your own dashboard data. Read-only, so you cannot break anything. Window functions, CTEs, margin without averaging ratios. Save queries.
Analysis Lab	`/analysis`	Descriptive, diagnostic, predictive and prescriptive analysis, with probability, time series, filtering and regression, on your own data.
Model Studio	`/studio`	Pick a table and a column to predict. The Studio tries many real models, picks the winner without peeking at the test rows, and tells you plainly if it cannot beat guessing.
ML Lab	`/ml`	Train a real scikit-learn model on the data and learn to judge it honestly: baseline, time-based split, leakage scan, threshold by cost.
ML by hand	`/mlhand`	Work small ML, language (NLP) and deep-learning problems step by step with every number shown, then check against the real library.
NLP and Deep Learning	`/nlp-dl`	Tell the agents what you need. They plan the job, run real models on your data, and compare against a do-nothing baseline.
AI Lab	`/ai`	An agent that answers questions by writing SQL, across Gemini, OpenAI and Claude. Includes usage counts, agent checks and evals.
Systems Design	`/design`	Describe a system in plain numbers. Get a first design, a cost estimate with the arithmetic shown, trade-offs, and a review of where it could fail.
Diagram studio	`/diagrams`	Type a flowchart as text and see it drawn. A small take on Mermaid and draw.io.

Lab	Address	What you do there
Practice vs Real	`/compare`	The same measures side by side.

Practice pages

Page	Address	What it is
Python practice	`/python`	Jupyter notebooks that load the dashboard's data, with exercises that check themselves.
Apache practice	`/apache`	Spark, Airflow and Superset, each wired to the dashboard's data.
Learn pipelines	`/pipeline`	The medallion pipeline: bronze, silver, gold.
Learn history tracking	`/scd`	Slowly changing dimensions: what happens to history when an entity's category changes.
Learn star schemas	`/schema`	How a fact table and its dimensions fit together.
Tool map	`/toolmap`	The tools people name in job ads: what the app has, what it stands in for, and what it deliberately does not embed.
Portfolio builder	`/portfolio`	What you have actually done, in a form you can show. Quiz and review scores are graded by the app; steps and case studies are your own record.
How-to guides	`/toolbox?role=howto`	Search how to do or fix anything.
How-to videos	`/howto`	Links to videos that show a job being done, sorted by field. A link opens in a new tab; nothing is downloaded.
Company engagement	`/company`	If you were hired to do every discipline for one company: the plan, the order, and where each piece gets done.

A weekly study routine that works

About five hours a week:

Slot	Time	Do this
Review	15 minutes, most days	Open **Review**. Clear what is due.
Learn	1 hour, twice a week	Open a track manual in Learn mode. Do one or two steps. Tick them only when the "how you know you are done" line is true.
Build	1.5 hours, once a week	Work a practice job from Clients (Practice workspace) in Work mode, using the field's tools.
Explain	30 minutes, once a week	Ask the field's agent to quiz you or to check your reasoning, then verify its answer yourself.

Slot	Time	Do this
Capture	15 minutes, once a week	Add what you did to the **Portfolio**, and write one line on what you got wrong.

Three rules for learning with an AI agent. Try the problem first. Ask the agent to explain the step, not to do it. Check any number it gives you by running the calculation yourself in the lab.

Study order

- **Work with data:** Analyst, Data Scientist, Machine Learning, then Data Engineering and AI Engineering.
- **Build software:** Full Stack, Systems Analyst, Data Engineering, then Cybersecurity.
- **Run and secure infrastructure:** IT repair, Network, Cybersecurity, then Full Stack.
- **Lead and organise work:** Systems Analyst, Project and Product, Analyst.
- **Not sure yet:** Analyst first (the best general foundation), then IT repair and Full Stack to sample the other worlds.

The companion book, *The Practitioner's Library*, teaches each of these fields from first principles with exercises. This manual tells you how to use the dashboard for them.

Part 6: The fields, one chapter each

How every field dashboard is built

Twelve fields have a **track dashboard** at `/tracks/<id>` (open it from **Tracks** in Learn mode or from **Fields** in either mode). Each has the same skeleton:

1. A row of buttons to related labs and pages.
2. The top search/agent box.
3. **How-to manual** (Learn mode only), **Tools**, **AI agent**.
4. In Work mode on a wide screen the agent sits beside the tools instead of in its own tab, and a collapsible **Fix log** lists what went wrong and what fixed it.
5. **Agents do the job:** pick an open job, then the field's agent and the three-AI team draft the work. A file upload is available for the job. Drafts only: nothing is sent, closed or marked done.
6. **Your data in every track:** a strip that checks whether your imported data is big enough for this field's exercises.
7. In Learn mode: the **Learning path** and **Portfolio projects** on the right, and "Build on it: practice ideas" under the tools.
8. For the eight fields with a shared advanced lab (below), a **Pro lab** at the bottom of the tools.

Each chapter below gives: what the field is, where its tools are, the manual steps in order, the advanced calculators, how to practise with a job, three portfolio projects, and the honest limits.

A note on honesty. Findings from security and network tools are leads, not verdicts. Calculators check your numbers but cannot know your real situation. AI answers can be wrong. A model that cannot beat a simple baseline is a bug, not a result.

Field 1: Data Analyst

What it is

Turn data into decisions people can act on. Analysts answer business questions with SQL, spreadsheets and BI tools, then explain the answer clearly.

Where to open it

Learn mode: **Tracks** > **Data Analyst** (`/tracks/analyst`). Work mode: **Fields** > **Data Analyst** (`/field/analyst`: pivots, charts, A/B reads, notebooks and data stories, plus the services you can sell).

The tools

- Stat tiles: **net margin** (profit divided by revenue, a ratio of sums), **cost vs budget** (positive means over), **completion**, and **data quality** score.
- Weekly net margin chart and a **Most over budget** table.
- **Analyst Pro Lab:** forecast with a range (checked against "repeat last value"), cohorts and retention (pasted or straight from a table), funnel, Pareto, A/B test, break-even, unit economics, margins, cash runway, cash cycle, NPV and IRR, star schema from a flat table, ten runnable window-function recipes, a SQL to DAX to Tableau to Excel map, dashboard design review, metric definition card, requirements interview and page, report QA checklist and self-service rollout.
- **BI toolkit:** a DAX lab, a forecast backtest, variance, scenarios, distributions and a document reader.
- **Survey panel:** check a survey percentage and read a joint probability table.

The how-to manual

The manual has 14 steps, in this order:

1. Frame the question
2. Understand the tables you query
3. Use the SQL and pandas cookbook
4. Check the data can be trusted
5. Learn the shape of the data
6. Define the KPIs
7. Investigate with SQL
8. Choose the right chart

9. Practise the BI building blocks (measures, forecasts, variance, documents)
10. Find the finding
11. Deliver the report
12. Monitor and repeat
13. Check a survey percentage before you quote it
14. Read a joint probability table

Learning path

read the dashboard like an analyst; SQL Lab exercises; data quality checks; notebooks `02_pandas_explore`, `10_analyst_case_study`, `11_hypothesis_testing`; build your own KPI; learn the vocabulary in the Glossary; take the quiz; then `docs/TABLEAU.md`, `docs/BI_TOOLS.md` and `docs/BI_TOOLKIT.md` for rebuilding the dashboard in other tools.

Do a practice job

In Practice, open an analyst job in Clients, read the brief, pin the job, run Data quality on the job file, answer the question in SQL Lab, chart it, write the one-page answer, then open the hidden answer key and compare.

Agent tools

Local calculations only. The field's agent can call: forecast, forecast range, cohort, funnel, unit economics, break-even, A/B test, variance, profit what-if, distribution fit and DAX.

Portfolio projects

1. A one-page memo "Where are we losing money?" with three charts and one recommendation
2. A Tableau Public dashboard
3. A SQL portfolio of ten queries each with its business question

Honest limits

Forecasts show a range because the future is uncertain. A/B results need enough sample. Tableau, Power BI and Excel are not embedded; the tool map says what the app stands in for.

A four-week route

Week	Do this
1	Open the Analyst track in Learn mode. Do manual steps 1 to 4: frame one question, read the tables, run the SQL cookbook, and run **Data quality** on the Practice data. Tick each step only when its "done" line is true.
2	Steps 5 to 8. Define two KPIs in the KPI builder, investigate one question in the SQL Lab, and make three charts from the Chart gallery. Say why you chose each chart type.
3	Steps 9 to 11. Try the Analyst Pro lab: a forecast with a range, a funnel, and a break-even. Write the one-page answer to your question.
4	Steps 12 to 14. Work an analyst practice job from Clients end to end, then open the answer key and write down every difference. Add the memo to the Portfolio.

Field 2: Data Scientist

What it is

Use statistics and modelling to find out what is true and what will happen, and be honest about uncertainty.

Where to open it

`/tracks/scientist` (Learn) or **Fields** > **Data Science** (`/field/science`).

The tools

the **weekend premium test** (Welch t-test on daily totals; each day is one data point, because records from the same day are not independent), **entity segments** (KMeans, k=3), a **correlation matrix** (Pearson), the BI toolkit, the survey panel, and in Learn mode the course panel.

The advanced lab

The Data Scientist Pro lab holds these calculators (each is also a tool for that field's agent):

- Bayesian A/B
- sample size
- Kaplan-Meier
- linear programming
- distance between places
- fairness audit
- difference in differences
- bootstrap interval

References in the same lab: references on causal inference, experiments and ethics.

The how-to manual

The manual has 17 steps, in this order:

1. Clarify the business question
2. Decide what data you need and where from
3. Write a testable hypothesis

4. Check the data is suitable
5. Prepare the data (the longest step)
6. Understand the tables you query
7. Use the SQL and pandas cookbook
8. Describe before you test
9. Run the test and read it properly
10. Segment and cluster
11. Model
12. Evaluate
13. Deploy
14. Feedback
15. Challenge your own conclusion
16. Write it up
17. Compare forecasting methods and check what shape your data has

Learning path

BI toolkit; notebooks `02_pandas_explore`, `05_forecast`, `11_hypothesis_testing`, `12_clustering_entities`; ML Lab; `13_ml_regression`.

Related pages

Analysis Lab (`/analysis`), Import data (`/data`), Notebooks (`/python`), ML Lab (`/ml`).

Portfolio projects

1. An A/B-style write-up with effect size, interval and what you would not claim
2. An entity segmentation with names and an action per segment
3. A forecast that beats seasonal-naive on a backtest

Honest limits

A strong correlation is not causation, and revenue against cost is strong partly by construction. A p-value is not the probability the claim is true.

A four-week route

Week	Do this
1	Manual steps 1 to 5: write the question and a testable hypothesis, then run **Data quality** and prepare the data. Read the weekend premium test and say what a p-value does not tell you.
2	Steps 6 to 9: describe before you test, run one test, and read its interval. Run the sample-size and bootstrap tools in the Pro lab.
3	Steps 10 to 12: segment the entities, name each segment, and evaluate a model against a baseline in the ML Lab.
4	Steps 13 to 17: write up one analysis including what you would not claim, and compare two forecasting methods on a backtest.

Field 3: Machine Learning

What it is

Build and evaluate models that survive contact with reality.

Where to open it

`/tracks/ml` or **Fields > ML, NLP and DL** (`/field/ml`, which also has a Computer vision pack).

The tools

the **experiment log** (every ML Lab run is logged here with its score, baseline and lift), the best result per task, MLOps panel (versions, release checks), model build, AI fundamentals, learn toolkit and course.

The advanced lab

The ML Pro lab holds these calculators (each is also a tool for that field's agent):

- convolution shape
- transformer size
- LSTM parameters
- exact Shapley values
- tuning budget
- item recommender
- forecast metrics (MASE)
- drift (PSI)
- serving capacity
- bandit simulation

References in the same lab: experiment tracking.

The how-to manual

The manual has 16 steps, in this order:

1. Learn how AI, ML and deep learning fit together
2. Define the target and the metric

3. Check you have enough data
4. Build the baseline first
5. Split by time, not randomly
6. Choose features
7. Train and compare
8. Log and review experiments
9. Decide: ship or not
10. Follow the learning path
11. See overfitting happen
12. Write the algorithms once, by hand
13. Pick the threshold with money, not 0.5
14. Scan for leakage before you trust a score
15. Check drift and read an automatic search honestly
16. Train a network, check its gradients, and look inside text models

Learning path

ML Lab; notebooks `13_ml_regression`, `14_ml_classification`, `15_mlflow_tracking`; then the source files `ml_lab.py`, `ml_tools.py`, `ml_build.py`, `learn_path.py` if you want to read how it works.

Portfolio projects

1. Predict "not completed" with a cost-aware threshold and justify it in dollars
2. An MLflow comparison of five models with a one-paragraph recommendation
3. Add a model to `ml_lab.py` with a test

Honest limits

Always compare with a do-nothing baseline. Random splits leak time. A high score on a small table is probably noise.

A four-week route

Week	Do this
1	Steps 1 to 4: define the target and metric, check you have enough data, and build the baseline in the ML Lab.

Week	Do this
2	Steps 5 to 8: split by time, choose features, train two models and compare them, and read the experiment log.
3	Steps 9 to 12: decide ship or not, watch overfitting happen, and write one algorithm by hand in ML by hand.
4	Steps 13 to 16: pick a threshold by cost, run the leakage scan, check drift with PSI in the Pro lab, and write one paragraph recommending a model.

Field 4: Data Engineering

What it is

Move data reliably from where it is made to where it is used.

Where to open it

`/tracks/engineering` or **Fields** > **Data Engineering** (`/field/eng`).

The tools

The tab list (five tabs):

- **Pipeline** (medallion run, history, health)
- **Data tools lab**
- **Database admin** (DBA: baseline, backup and verified restore, benchmark and query plans, checkpoint and growth)
- **Design** (architecture and warehouse design)
- **Data governance** (owners, classification, personal data, lineage, controls)

The advanced lab

The Data Engineer Pro lab holds these calculators (each is also a tool for that field's agent):

- Kafka sizing
- Spark partitions and skew
- DAG validator
- data contract check
- volume anomaly
- warehouse cost
- small-files compaction

References in the same lab: lakehouse, streaming and observability references.

The how-to manual

The manual has 35 steps, in this order:

1. The pipeline (understand bronze, silver and gold
2. Run it
3. Read the quarantine
4. Prove it idempotent
5. Bring in real data)
6. DBA (health baseline, back up and prove the restore, benchmark and read plans, checkpoint and watch growth)
7. Architect (draw the flow, choose the store)
8. Warehouse engineer (design a star schema, plan loads and checks)
9. Manager (data management plan, analytics-ready, metadata)
10. Then ETL versus ELT, wrangling, security, performance, dbt-style models, spreadsheets as formula graphs, scheduling and streams, core queries, compliance map, DataOps sprint, table scores, ERD design, change capture, data issue queue, ecosystem check, and four governance steps

Portfolio projects

1. Make `medallion.py` read from an API with the same watermark logic
2. Add a freshness test that fails when the newest day is stale
3. Wrap the pipeline in an Airflow DAG
4. Write a one-page backup and recovery plan with RPO and RTO and prove it by restoring
5. Classify every table and get the governance checklist to eight of eight

Honest limits

A backup you have not restored is a hope. Spark, Airflow and Kafka are taught and sized here, but the app runs DuckDB and Parquet locally.

A four-week route

Week	Do this
1	Pipeline steps: understand bronze, silver and gold, run the pipeline, read the quarantine, and prove it is idempotent.
2	DBA steps: take a health baseline, back up, **restore the backup and verify it**, benchmark and read a plan.
3	Architect and warehouse steps: draw the flow from sources to users, design a star schema, and plan the loads and checks.
4	Governance steps: classify every table, find personal data, document lineage. Try the Pro lab's data contract check and volume anomaly.

Field 5: AI Engineering

What it is

Build dependable products on top of language models.

Where to open it

`/tracks/ai` or **Fields > AI engineering** (`/field/ai`).

The tools

Tools

- provider status tiles (used today against the daily limit, key variable names)
- usage so far
- recent questions
- **Prompt library**
- **AI workbench**
- **Governance** (register AI assets, classify a use case by risk, policy and spend rules)
- **Implementation** (is a workflow ready for AI, autonomy level, vendor scoring)
- MLOps
- **Inference** (size and cost a model)
- Python libraries
- course
- and the learn toolkit

The advanced lab

The AI Pro lab holds these calculators (each is also a tool for that field's agent):

- LoRA memory
- vector database size
- retrieval metrics
- chunking
- quantization speed
- KV cache

- LLM cost with caching
- prompt comparison

References in the same lab: red team, guardrails, LLMOps and multimodal references.

The how-to manual

The manual has 28 steps, in this order:

1. Check your own AI work before shipping
2. Release a model like an engineer
3. Choose the right approach before you build
4. Check providers and keys
5. Set the privacy mode first
6. Ask a data question and read the steps
7. Verify answers independently
8. Understand routing, cost and limits
9. Test prompt injection
10. Size and cost a model before you run it
11. Build the pieces yourself
12. Follow the nine-level path
13. Measure retrieval before blaming the prompt
14. Never let unvalidated output through
15. Read an agent trace like a post-mortem
16. Know how sure your eval score is
17. Reliability arithmetic
18. Training data and run size
19. Redact and scan before text reaches a model
20. Register AI assets
21. Classify use cases by risk
22. Govern actions and spend
23. Tour the Python libraries
24. Decide if a workflow is ready
25. Set the autonomy level
26. Score an AI vendor
27. Choose free courses
28. See how tokens and attention work

Learning path

AI Lab; `ai_engineering/01_structured_output.py`, `02_rag.py`, `03_evals.py`, `04_mcp_server.py`, `05_prompt_injection.py`; then the agent, router, providers and tools source files.

Portfolio projects

1. Grow the eval to thirty questions and track the score across prompt versions
2. Swap TF-IDF for embeddings in `02_rag.py` and compare hit-rate
3. Connect the MCP server to Claude Desktop and record a demo

Honest limits

The agent plumbing is tested with a scripted fake model. Real model quality depends on the provider. Live provider calls have not been exercised by the build, so verify answers, and remember an eval score tests structure, not truth.

A four-week route

Week	Do this
1	Check providers and keys, set the privacy mode, ask a data question and read the steps, and verify the answer independently.
2	Understand routing, cost and limits. Test prompt injection. Size and cost a model in the Inference panel.
3	Run `01_structured_output.py` to `03_evals.py`. Measure retrieval before blaming the prompt. Use the Pro lab's retrieval metrics and chunking tools.
4	Governance: register an AI asset, classify a use case by risk, and score a vendor. Write your own eval of ten questions and score it.

Field 6: Project and Product Management

What it is

Decide what to build, in what order, on time and on budget.

Where to open it

`/tracks/pm` or **Fields > Project management** (`/field/pm`, with an IT project management pack).

The tools

The tab list (13 tabs):

- **Board**
- **Prioritisation** (RICE and value scoring)
- **Sprints**
- **Flow** (cumulative flow and cycle time)
- **Schedule** (Gantt and critical path; drag to resize a task)
- **Earned value** (CPI and SPI)
- **Risks** (expected cost, contingency)
- **OKRs**
- **Product analytics**
- **Time**
- **What-if**
- **Calendar**
- **Automations**

A **lead** panel supports your own pipeline.

The advanced lab

The Project Manager Pro lab holds these calculators (each is also a tool for that field's agent):

- Monte Carlo schedule risk
- critical path

- PERT
- RACI check
- stakeholder map
- resource loading
- earned value status report
- pricing

References in the same lab: PMBOK, retrospectives, roadmapping, agile, contracts and change references.

The how-to manual

The manual has 12 steps, in this order:

1. Set the goal as OKRs
2. Build the backlog
3. Prioritise by value
4. Plan the schedule and find the critical path
5. Plan a sprint
6. Run the work and watch flow
7. Track budget with earned value
8. Manage risks
9. Measure the product
10. Review and adjust
11. Run a self-review on five leadership disciplines
12. Resize a task on the Gantt chart and draw a process on the canvas

Agent drafts

backlog items, risks and OKRs. Press Add on the card to save them.

Portfolio projects

1. Run a real two-week project (ten items with RICE scores, one sprint, one burndown, a retro note)
2. A one-page PRD
3. A risk register with expected costs and a contingency reserve you can defend in two sentences
4. Explain CPI and SPI to a sponsor without the acronyms

Honest limits

The board is a personal-scale tracker, not Jira for a team. A Monte Carlo result is only as good as your three-point estimates.

A four-week route

Week	Do this
1	Set the goal as OKRs and build a backlog of ten items.
2	Prioritise with RICE, then plan the schedule and find the critical path. Try the Monte Carlo tool in the Pro lab.
3	Plan and run a one-week sprint on the board and watch the flow tab.
4	Track budget with earned value, build a risk register with expected costs, and write a retro note.

Field 7: Systems Analyst

What it is

Work out what a system must do and prove it does.

Where to open it

`/tracks/sysanalyst`.

The tools

The tab list (6 tabs):

- **Requirements** (with acceptance criteria and links to tests)
- **Data dictionary**
- **Process analysis** (where time goes, the bottleneck)
- **Calculators** (sizing and queueing)
- **Feasibility (TELOS)** (technical, economic, legal, operational, scheduling)
- **Single source of truth** (which numbers matter and how systems connect)

The advanced lab

The Systems Analyst Pro lab holds these calculators (each is also a tool for that field's agent):

- user stories
- Mermaid diagrams
- process lint
- UAT cases
- migration mapping and cutover checklist
- vendor scoring
- change impact
- business case
- data requirements
- integration pattern

References in the same lab: TOGAF-lite and elicitation references.

The how-to manual

The manual has 10 steps, in this order:

1. Write the problem statement and list stakeholders
2. Capture requirements
3. Model the data
4. Model the process
5. Size the system
6. Make the cost-benefit case
7. Score feasibility (TELOS)
8. Trace requirements to tests
9. Audit how systems connect and which numbers matter
10. Audit your documents and key-person risk

Learning path

read the data model; check the data is fit to model; verify requirements with SQL; model change over time; trace the data flow.

Portfolio projects

1. Ten testable requirements for a system you know, each linked to a test
2. The as-is process drawn from data
3. A TELOS feasibility memo with the first risk you would raise
4. A service desk sized for an average wait under five minutes

Honest limits

Requirements come from people, not from the app. The tool checks that they are testable, not that they are right.

A four-week route

Week	Do this
1	Write the problem statement and list stakeholders. Capture five requirements with acceptance criteria.
2	Read the data dictionary and model the data. Model one process and find its bottleneck.

Week	Do this
3	Size the system with the Calculators tab, and write a short cost-benefit case.
4	Score feasibility with TELOS, trace each requirement to a test, and audit the documents and key-person risk.

Field 8: Full Stack Developer

What it is

Own a feature from the table to the screen.

Where to open it

`/tracks/fullstack` or **Fields** > **Full stack** (`/field/fs`, with a DevOps pack).

The tools

The tab list (7 tabs):

- **Code Workbench**
- **Generate code** (describe it and get checked, risk-scanned code to copy; it is never run)
- **API map & tester** (every endpoint, try one, see the timing)
- **Scaffold from a table** (model, router, test and TypeScript type)
- **Codebase**
- **Stack & config**
- **Software engineering**

The advanced lab

The Full-stack Pro lab holds these calculators (each is also a tool for that field's agent):

- JWT checker
- OpenAPI lint
- Dockerfile review
- CI workflow starter
- migration lint
- Core Web Vitals
- accessibility check and contrast
- cache maths
- queue sizing
- password hash cost

References in the same lab: auth, testing, observability, API design, git and system-design references.

The how-to manual

The manual has 10 steps, in this order:

1. Plan the work (requirements, method, tests)
2. Map the API you have
3. Design the resource
4. Scaffold the layers
5. Register and run it
6. Try it and break it
7. Write real tests
8. Build the screen
9. Review code health
10. Run the release checklist

Portfolio projects

1. Scaffold a table, register the router and add a test that fails when it should
2. Add a field end to end (column, API model, TypeScript type, form input, test)
3. Find the slowest endpoint with the API tester and show before and after
4. Write and run a release checklist

Honest limits

Generated code is a draft: read it, test it, and never run code you do not understand. The tester talks to this app only.

A four-week route

Week	Do this
1	Plan the work, then map the API you have in the API map and tester.
2	Design a resource and scaffold it from a table. Register it and run it.
3	Try it and break it. Write real tests, including one that fails when it should.
4	Build the screen, review code health, and run the release checklist. Use the Pro lab's OpenAPI lint and Dockerfile review on your own files.

Field 9: Cybersecurity

What it is

Protect systems you own: find weaknesses, read logs, respond.

Where to open it

`/tracks/security` or **Fields** > **Cybersecurity**. A related field, **DevSecOps** (`/field/devsecops`), covers pipeline and cloud security and revenue-system security.

The tools

The tab list (16 tabs):

- **Checkup report**
- **How to fix**
- **Fix log**
- **Client records**
- **Phishing and breach**
- **Mac and Linux audit**
- **Self-audit**
- **Log analysis**
- **Web checks** (headers and TLS)
- **Local ports**
- **Endpoint hardening**
- **Honeypot logs**
- **Principles and NIST CSF**
- **Defender tools**
- **Crypto labs** (hashes, passwords, 2FA, tokens)
- **Incidents**

In Work mode a **Known issues** panel shows CISA's list of flaws attackers are using (downloaded once a day and searched on your computer; it falls back to the last saved copy with its date if the site cannot be reached).

The advanced lab

The Security Pro lab holds these calculators (each is also a tool for that field's agent):

- CVSS 3.1 calculator
- annualized loss and control payback
- risk matrix and register
- pen-test scope template
- OWASP Top 10
- attack stages
- forensic timeline
- Suricata and Snort rule explainer
- and HIPAA
- SOC 2
- PCI and continuity checklists

The how-to manual

The manual has 11 steps, in this order:

1. Audit your own setup
2. Read the logs for patterns
3. Check what a site shows the world
4. Know what your machine exposes
5. Understand hashes, passwords and 2FA
6. Handle an incident on paper
7. Read honeypot logs like an analyst
8. Harden a Windows endpoint, one measured change at a time
9. Judge a design against the five principles and the CIA triad
10. Run a NIST Cybersecurity Framework self-check
11. Inspect a token, score a flaw, model a threat, test a rule

Portfolio projects

1. Run the self-audit, fix every red item and run it again
2. Paste the sample auth log, find the brute-force attempt and write the incident report
3. Check the security headers of your own site
4. Write an incident playbook for "laptop lost" and run it as a tabletop exercise

Rules that never bend

Defensive work only. Your own systems or written permission. Offensive and cloud-attack tools are taught about, not built. A detection finding is a lead and not a verdict. A guardrail is not a vault. Computer control (`/computer`, shown when enabled) runs a fixed list of actions on this computer behind a PIN with undo; arm it only when you mean to.

A four-week route

Week	Do this
1	Run the **Self-audit** on your own setup and fix every red item.
2	Read the logs for patterns in the sample auth log, check what your own site shows with the Web checks, and review local ports.
3	Hashes, passwords and 2FA in the Crypto labs. Harden one Windows setting at a time and measure it.
4	Handle an incident on paper, run the NIST CSF self-check, score one flaw with the CVSS calculator, and write an incident playbook.

Field 10: Network Engineer

What it is

Design, build and fix the paths data takes.

Where to open it

`/tracks/network` or **Fields** > **Network**.

The tools

The tab list (11 tabs):

- **Subnet and VLSM** (subnets, summaries, overlaps, contains, split)
- **Fix log**
- **Config review** (paste a switch or router config and read the findings)
- **Packet capture**
- **DNS and reachability**
- **Config generators**
- **Network design**
- **Simulators** (switch, spanning tree, OSPF, BGP, ACL, NAT)
- **Capture file reader** (open a saved `.pcap` and see what looks wrong)
- **Calculators** (transfer time, bandwidth-delay, bandwidth)
- **Reference**

The Network Pro lab and a Cloud lab sit below.

The advanced lab

The Network Pro lab holds these calculators (each is also a tool for that field's agent):

- OSPF cost
- spanning tree root
- IPv6 plan
- EUI-64
- PoE budget

- fiber budget
- wireless link budget
- TCP speed limit
- firewall rule review
- VPN
- QoS
- and an SNMP and monitoring reference

The how-to manual

The manual has 12 steps, in this order:

1. Plan the addressing
2. Review a device config
3. Read what is on the wire
4. Test the path
5. Watch the protocols decide (switch, spanning tree, OSPF, BGP, ACL, NAT)
6. Read a saved capture and find what looks wrong
7. Size the pipe
8. Keep a method and a reference
9. Build and check VLANs, trunks and routes
10. Set up a DHCP and DNS server (dnsmasq)
11. Design a small network on paper before buying anything
12. Learn the networking vocabulary, protocols and default ports

Use live capture with the reader

Start with launcher choice 2, capture 30 seconds on your own network, download the `.pcap`, then open it in the **Capture file reader**. A typical question to answer: is this a scan, normal monitoring or a broken client? List the next two things you would check.

Portfolio projects

1. Plan a small office in one /22 and check for overlaps
2. Fix every high finding in the sample switch config and explain each in one sentence
3. Read the sample capture and decide what it is
4. Write a one-page "the internet is down" runbook that walks the layers

Honest limits

Tools run on pasted text and on public hosts. Nothing here scans networks you do not own. The simulators teach the idea; real devices differ.

A four-week route

Week	Do this
1	Plan the addressing for a small office in the Subnet and VLSM tab and check for overlaps.
2	Paste the sample switch config into Config review and fix every high finding. Test the path in DNS and reachability.
3	Run the simulators (spanning tree, OSPF, ACL, NAT). Read a saved capture and say what looks wrong.
4	Size a pipe with the calculators, design a small network on paper, and write the "internet is down" runbook.

Field 11: IT Repair and IT Specialist

What it is

Keep people productive, protect data, know what you own. This is the field closest to your repair services.

Where to open it

`/tracks/itsupport` or **Fields > IT repair**.

The tools

The tab list (16 tabs):

- **Helpdesk** (tickets with SLA clocks)
- **Fix log**
- **How to repair** (repair guides, including eleven admin guides)
- **Inventory** (assets and warranty dates)
- **Backup lab** (incremental backup and prove the restore)
- **Event logs** (Windows events, paste and rank)
- **Checklists**
- **Capacity and RAID**
- **Cheat sheets**
- **Windows shortcuts**
- **Software guide**
- **Virtualization (vSphere)**
- **CompTIA A+**
- **IT fundamentals**
- **Infrastructure labs**
- **Ticket coach**

Below the tabs: **IT Admin lab** and a **Cloud lab**, plus the **Known issues and support dates** panel in Work mode (CISA exploited-flaw list and endoflife.date support dates). **IT Admin lab:** email authentication check (SPF, DKIM, DMARC), Microsoft 365 and Intune reference, onboarding and offboarding runbooks, script review, priority matrix, SLA maths, stop codes, patch policy.

The how-to manual

The manual has 18 steps, in this order:

1. Work a ticket properly
2. Know what you own
3. Read the event log
4. Use checklists for routines
5. Forecast and protect
6. Plan a vSphere build
7. Choose tools on purpose
8. Study A+ Core 1
9. Study A+ Core 2
10. Learn the fundamentals (numbers, permissions, the command line)
11. Build your own toolbox
12. Predict which Group Policy wins
13. Read and write LDAP data
14. Size a RAID, a backup scheme and a recovery plan
15. Triage a website and a load balancer
16. Write tickets that get fixed first time
17. Take an incremental backup and prove you can restore it
18. Collect an inventory from real machines with an agent script

Portfolio projects

1. Log five tickets, work each checklist, resolve them and read the mean time to resolve
2. Enter your own devices and list warranties ending this year
3. Paste the sample event log, rank the findings and say what you would do first
4. Write onboarding and offboarding checklists for a five-person business and run both once
5. Rate every A+ objective, quiz your weakest domain, and re-rate

Honest limits

RAID is not a backup. The helpdesk and inventory are personal-scale trackers. For certifications, check the current exam objectives. Practice jobs for this field have no made-up files: use your own equipment.

A four-week route

Week	Do this
1	Log five tickets in the Helpdesk and work each with its checklist. Read the mean time to resolve.
2	Enter your own devices in the Inventory. Read the event-log sample and rank the findings.
3	Do the Backup lab, including the restore. Size a RAID. Use the IT Admin lab's email authentication check.
4	Study one A+ domain and take a quiz on it. Write onboarding and offboarding checklists and run both.

Field 12: Computer Engineer

What it is

How computers, circuits and vehicles work underneath.

Where to open it

`/tracks/cpe` or **Fields** > **Computer engineering** (`/field/cpe`).

The tools

The tab list (the tab groups):

- **Numbers** (number systems, signed add and overflow, floating point)
- **Logic** (truth table and simplify, are two expressions equal, logic circuit)
- **Circuits** (Ohm's law and power, series and parallel, voltage divider, LED resistor, RC charge time, resistor colour code, circuit solver, charge and discharge)
- **Architecture** (cache address split, average memory access time, CPU time, Amdahl's law, pipeline speedup, tiny CPU, cache trace, RAID capacity)
- **Vehicle wiring** (wire, drop and fuse, harness checker, CAN bus load and termination, CAN signals, J1939 messages, trouble codes, OBD-II readings, diagnostic frames)
- **EV** (AC and DC charge time, battery pack layout, pre-charge circuit, insulation check, charge pilot duty, charge pilot voltage, plug proximity)
- and a **simulator** group

Below: a **PC build lab**, a **robotics lab**, an **electrical and HVAC lab**, and a **computer science lab**.

The advanced lab

The Computer Engineer Pro lab holds these calculators (each is also a tool for that field's agent):

- state machine check with Verilog
- setup and hold timing
- pipeline CPI
- virtual memory and TLB
- average memory access time
- UART baud

- timer prescaler
- I2C pull-ups
- ADC resolution and ENOB
- op-amp gain
- RC filter and order
- buck and boost design
- DC motor operating point
- second-order response
- PID from Ziegler-Nichols
- PCB trace width
- microstrip impedance
- reliability and MTBF
- derating

References in the same lab: datasheet, embedded, HDL and FPGA, architecture, bench, standards and PCB-flow references.

The how-to manual

The manual has 10 steps, in this order:

1. Move between binary, decimal and hex, and see overflow
2. Simplify logic and prove two circuits are the same
3. Size a resistor and read a simple DC circuit
4. Read a cache address and judge a speedup
5. Choose the wire and fuse for a 12 V circuit
6. Know how the main vehicle electronic systems fit together
7. Plan EV charging and understand high-voltage basics
8. Simulate a circuit, a CPU and a cache
9. Decode what a vehicle says on its network
10. Apply it: a small vehicle conversion on paper

Portfolio projects

1. Convert five numbers by hand and overflow two 8-bit adds
2. A truth table for a three-input alarm, simplified, drawn with NAND gates
3. Wire and fuse for six circuits checked with the harness checker
4. CAN bus load for ten messages

5. A home charging plan with 20 to 80% time for your car

Safety

High-voltage battery systems can kill. The EV tools are for planning and learning on paper. Do not work on live high-voltage systems without proper training and equipment.

A four-week route

Week	Do this
1	Number systems and signed add: convert by hand, then check. Truth tables and simplification.
2	Ohm's law, a voltage divider, an LED resistor and an RC time constant. Check each against the circuit solver.
3	Architecture: cache address split, average memory access time, pipeline speedup. Use the simulators.
4	Vehicle wiring: size the wire and fuse for six circuits, estimate a CAN bus load, plan home charging. Treat high voltage as paper-only.

The other fields

These open as a **field page** (`/field/<name>`) with: the Agents-do-the-job card, a **Fix log**, **Services you can sell in this field** (price range and hourly rate), and the field's tools.

Field	Address	Tools and packs
BI	`/field/bi`	BI tools, KPI dictionaries, scheduled reports, plus a **Revenue operations** pack
Cloud	`/field/cloud`	Cloud architecture sizing, cost estimates, design write-ups
DevSecOps	`/field/devsecops`	**DevSecOps** pack (pipeline and cloud security checks) and **Revenue-system security**
Health data	`/field/health`	**Clinical data science**, **Health analytics**, **Health BI**, **Health informatics**. Made-up or de-identified data only.
Science and research	`/field/research`	**Statistics**, **Genomic data science**, **Science and research**
Business consulting	`/field/consulting`	Assessments, strategy documents, leadership and process tools
Marketing and design	`/field/marketing`	**Digital marketing**, **UI/UX and graphic design**

Use a pack the same way as any tool: open the pack, fill in the numbers or paste the text, read the result, and keep what you decide in the job's notes. For health data, never put real patient information into the dashboard, and never into an AI question.

Part 7: AI, privacy and trust

Setting up the AI services

Put at least one key in `backend/.env` (copy `.env.example` to `.env`). The variable names are `GOOGLE_API_KEY` (Gemini), `OPENAI_API_KEY` and `ANTHROPIC_API_KEY` (Claude). Restart the backend after changing the file. Check the status tiles on an AI dashboard: each shows used-today against the daily limit, or "not set up". Without a key everything except the AI features still works, and the agent says so plainly. The limits are 10 AI questions per minute (shared with the AI Lab) and a daily cap per provider. Daily counts reset when the backend restarts.

Never paste a key into a chat, a ticket or a file you will share. Keys live in `.env` only.

Privacy modes

Set per workspace in **Settings > AI and privacy**.

Mode	What the AI can see
Off	Nothing is sent to any AI provider. The Real workspace starts here.
Summaries only	Counts, totals, averages and scores. At most 20 result rows. No titles or names.
Full	Any read-only query, up to 40 rows; titles are included.
Blocked columns	Hidden from the AI's view of the schema, rejected in its queries and rejected if they appear in a result.

These are **guardrails, not a vault.** They stop the assistant from reading what you blocked through its normal tools and they stop accidents. Whatever it is allowed to see goes to the provider you chose. A column could still leak if you rename it in a view or paste its values into your question. If data must never leave your computer, keep AI off.

What the AI features are, and how to use each

Feature	Where	How to use it well
Field agent	Every track and field page	Ask a question, give an instruction, or press Ask on a manual step. Read the trace. Verify numbers.
Top search box	Every field page	"Take me to the schedule tab" or "draft 5 requirements for a billing portal".
AI Lab	`/ai`	Ask a data question and read the SQL it wrote. Check usage, run Agent checks.
Work team	Pinned job bar	Three drafts plus a judged final draft, into the Assistant desk. Off by default, with a daily cap.

Feature	Where	How to use it well
Prep drafts	Each work item and each table	Optional AI first look. OpenAI only, off by default, daily cap 1 to 50 (default 10). No client names sent.
Send to agent	Web panel on any dashboard	Select text on a page and send it to the right agent as a labelled outside source (up to 4,000 characters, passwords blanked). Save the answer to the fix log as "Not fixed yet" until you confirm it works.
Generate code	Full Stack	Describe it; get checked, risk-scanned code to copy. Never run by the app.
Agent checks	AI Lab	The past-work lookup check is free and uses sample data. The live field-work check makes real calls (up to nine questions) against a keyword rubric. It is a floor, not a grade.

Seven habits that keep AI useful

1. **Say what you want checked.** "Review this config" gets a better result than "is this ok".
2. **Give it only what it needs.** Strip client names, addresses and secrets.
3. **Treat every number as a claim.** Re-run it in the SQL Lab or the calculator.
4. **Ask for the reasoning, then judge it.** If the reasoning is wrong the answer is too.
5. **Use drafts as drafts.** Edit them, then send them yourself.
6. **Read the trace.** It shows which tools the agent actually called.
7. **Keep your own fix log.** The IT, Cybersecurity and Network agents read your past fixes (Full mode) and get better at your problems, not at general ones.

Part 8: When something goes wrong

Troubleshooting table

Symptom	Likely cause	What to do
Page loads but shows errors or "can't reach the API"	Backend not running	Launcher choice 1, or start the backend terminal; check `http://localhost:8020/health`
A new page or tool is missing after an update	Screen not rebuilt, or backend not restarted	`npm run build` in `frontend`, restart the backend
Launcher says it started but nothing opens	Build still running or a port in use	Launcher choice 3 (stop), wait, choice 1 again
Two backends fighting	Both WSL and Windows backends started	Launcher choice 3, then start only one
Capture says no interfaces or permission denied	Not running as administrator, or Npcap missing	Use launcher choice 2 and accept the administrator prompt; confirm Npcap is installed
Capture works but Wireshark shows odd gaps (older captures)	Pre-zip-54 truncation	Install zip 54 and capture again
Scanned PDF or photo returns no text	Tesseract not available to this backend	Launcher choice 1 uses the WSL copy. For choice 2, install Tesseract for Windows (Part 4) and restart the backend
An upload is refused with a plain message	File rules (`.crdownload`, empty, old `.xls`, `.evtx`, zip)	Read the message; re-download or convert (save `.xls` as `.xlsx`)
AI says "not set up"	No key, or key not loaded	Add the key to `backend/.env`, restart the backend
AI refuses to answer	Privacy mode Off, or daily cap reached	**Settings > AI and privacy**; wait for the reset (restart) or raise the cap
A refresh was refused	The new data had under 50% of the old rows	Check the export; the old table is untouched
Numbers look wrong on a dashboard	Averaging ratios, or bad data	Run **Data quality**; check that margin is profit divided by revenue
Real data seems empty	You are in Practice	Check the Practice / Real switch at the top
Git push includes data you did not mean to commit	Real uploads in the repo	Run `git status` before committing; `.gitignore` covers the warehouse and intake folders

Install and update checklist

1. Download `bi_updateN.zip` to the project folder.
2. Run the install commands (Part 1).
3. `git status` and read the list before `git push`.
4. Rebuild the screen, restart the backend.
5. Open the dashboard, check the version in **Ops monitor**, take a backup.

Backups, restores and a recovery drill

Once a month do a drill: download a backup, restore it in the Practice workspace, run **Data quality**, and confirm the row counts. A backup you have never restored is a hope, not a plan.

What is not built or not tested

Be clear with yourself about these.

- **Not live-tested:** every AI provider call. Plumbing is tested with a scripted fake model.
- **Not tested on Windows:** the launcher .bat was proofread only.
- **Not written:** quizzes for the newest advanced labs; the optional "by hand" tabs for NLP and deep learning; a security review against your own transcripts.
- **Held on purpose:** Gmail and Calendar sign-in (until you say ready), and hosting online (which would need login, HTTPS and a strong owner password first).
- **Not embedded:** Tableau, Power BI, Jira, Splunk, Wireshark, Airflow clusters and the like. The Tool map names what the app stands in for and what it does not.
- **Single user, single machine:** no multi-user access control, no row-level security, no encryption of files on disk.

Appendix A: Every page and where it lives

Page	Address	Mode
Home	`/`	both
Clients, job page	`/clients`, `/job?id=`	Work
Money	`/money`	Work
Leads	`/leads`	Work
My data	`/data`	Work
Data Intake	`/intake`	Work
Data quality	`/quality`	Work
Dashboard	`/bi`	Work
Business dashboards	`/packs`	Work
KPIs	`/kpis`	Work
Company	`/company`	Work
Sources	`/sources`	Work (More)
Pipelines	`/pipelines`	Work (More)
Workflow builder	`/workflow`	Work (More)
Dataset hub	`/hub`	Work (More)
Sample work page	`/samples`	Work (More)
Knowledge search	`/knowledge`	Work (More)
How-to videos	`/howto`	Work (More)
Triage checklists	`/checklists`	Work (More)
Ops monitor	`/ops`	Work (More)
Log explorer	`/logs`	Work (More)
Activity log	`/activity`	Work (More)
Settings	`/settings`	both
Computer control	`/computer`	when enabled
Fields (main screen)	`/fields`	both
A field page	`/field/<name>`	both

Page	Address	Mode
A track dashboard	`/tracks/<id>`	both
Tracks list	`/tracks`	Learn
Quiz, Review, Glossary	`/quiz`, `/review`, `/glossary`	Learn
SQL Lab	`/lab`	Learn
Analysis Lab	`/analysis`	Learn
Model Studio	`/studio`	Learn
NLP and Deep Learning	`/nlp-dl`	Learn
ML Lab, ML by hand	`/ml`, `/mlhand`	Learn
AI Lab	`/ai`	Learn
Systems Design	`/design`	Learn
Diagram studio	`/diagrams`	Learn
Practice vs Real	`/compare`	Learn
Python practice	`/python`	Learn
Apache practice	`/apache`	Learn
Learn pipelines, history tracking, star schemas	`/pipeline`, `/scd`, `/schema`	Learn
Tool map	`/toolmap`	Learn
Portfolio	`/portfolio`	Learn
How-to guides	`/toolbox?role=howto`	Learn

Track ids for `/tracks/<id>`: `analyst`, `scientist`, `ml`, `engineering`, `ai`, `pm`, `sysanalyst`, `fullstack`, `security`, `network`, `itsupport`, `cpe`.

Appendix B: Quick glossary

Term	Meaning
Agent	An AI helper tied to a field. It reads, calculates and drafts. It cannot change anything by itself.
Baseline	The do-nothing answer a model must beat, such as "repeat the last value".
Bronze, silver, gold	Raw, cleaned and ready-to-use layers of a data pipeline.
CVSS	A score for how severe a software flaw is.
Fix log	Your own record of what went wrong and what fixed it, searched by the IT, Security and Network agents.
Guardrail	A rule that stops accidents. Not a vault.
Idempotent	Running a step twice gives the same result as once.
KPI	A metric tied to a target.
Lead (security)	A finding worth checking. Not a verdict.
Leakage	Information in a model's inputs that would not exist at prediction time, inflating its score.
OCR	Turning a picture of text into text.
Practice / Real	The made-up workspace and your own-data workspace.
Quarantine	Where a pipeline puts rows that failed its checks.
Ratio of sums	The correct way to get a margin: total profit divided by total revenue.
RPO and RTO	How much data you can afford to lose, and how long recovery may take.
SLA	A promised response or fix time.
TELOS	Technical, economic, legal, operational and scheduling feasibility.
Work / Learn mode	The earning menu and the studying menu.

Appendix C: Your first month

Week	Do this
1	Run the launcher. Learn the layout (Part 1). Switch between Practice and Real once. Take two quiz rounds. Take a backup and download it.
2	In Practice, work three practice jobs end to end: top box, Start as job, timer, field tools, Close job. Open the answer keys afterwards.
3	Pick one field. Do its manual steps one to five in Learn mode and tick them only when "done" is true. Add a line to the Portfolio.
4	Switch to Real with one small real job. Use the timer, close the job, write the invoice, read Money. Write down what annoyed you. That list decides what is built next.